SanaVerkko: Codebase and Algorithms

with Historical Background on Gematria, Numerology, POS, and Neural Language Models

Project Documentation

March 9, 2026

Contents

1 Introduction **2**

2 Codebase Architecture and Main Components **3**
2.1 Primary modules . . . 3
2.2 Execution model . . . 3

3 Core Algorithms in SanaVerkko **4**
3.1 Gematria and numerological features . . . 4
3.2 Indexed candidate retrieval . . . 4
3.3 Strict vs fluid gematria behavior . . . 4
3.4 POS matching and fluid POS . . . 5
3.5 Candidate scoring and selection . . . 5
3.6 Sentence iteration and convergence guard . . . 5
3.7 Audio synthesis algorithm . . . 5

4 Gematria and Numerology in Historical Perspective **6**
4.1 From ancient number-symbolism to formal systems . . . 6
4.2 Hebrew Kabbalah and numerological interpretation . . . 6
4.3 Hermetic Qabalah (based on `chicken_qabalah.txt`) . . . 6

5 Part-of-Speech (POS) Tagging **7**
5.1 POS in NLP . . . 7
5.2 POS implementation in SanaVerkko . . . 7

6 Neural Language Models: General and SanaVerkko-Specific **8**
6.1 General overview . . . 8
6.2 SanaVerkko LTM model architecture . . . 8
6.3 Training objective . . . 8
6.4 Inference-time integration in SanaVerkko . . . 8
6.5 GPU acceleration path . . . 9

7 Discussion and Future Directions **10**
7.1 Strengths . . . 10
7.2 Limitations . . . 10
7.3 Potential extensions . . . 10

Source Notes **11**

Chapter 1

Introduction

SanaVerkko is an experimental system that combines symbolic text transformation, numerological feature engineering, part-of-speech (POS) constraints, and real-time audio synthesis. The project couples a simulation/control loop with a reference database of words and a ranking model that can optionally be guided by a learned long-term memory (LTM) language model.

At a high level, each word in an active sentence is represented as a node with an activation value. When activation crosses configurable thresholds, the word may be replaced by another reference word according to gematria-based matching, optional POS constraints, optional exploratory jump logic, and optional LTM-based re-ranking.

Chapter 2

Codebase Architecture and Main Components

2.1 Primary modules

The codebase is centered around three primary modules:

- **sanaVerkkoCore.py**: main controller, UI logic (wxPython), simulation loop, replacement logic, indexing, and integration with LTM.
- **sanasyna.py**: audio engine and waveform rendering; melody generation, ADSR shaping, and multi-voice counterpoint mixing.
- **sv_ltm.py**: trainable next-word model (Char-CNN + MLP + softmax), serialization to `.svltm`, and inference API used by the core app.

2.2 Execution model

The runtime model is event-driven:

1. UI parameters are set in wxPython controls.
2. A timer calls `simulationStep()` periodically.
3. Word activations update and trigger replacement checks.
4. Updated sentence state is rendered and written to output.
5. Audio state is regenerated when sentence/activation signature changes.

Chapter 3

Core Algorithms in SanaVerkko

3.1 Gematria and numerological features

For a word $w = c_1 c_2 \dots c_n$, gematria is computed as

$$G(w) = \sum_{i=1}^{n} v(c_i),$$

where $v(c_i)$ is the configured numeric value of character c_i.

Two derived features are used:

- **Reduction**: one-pass digit sum (e.g., $137 \to 11$).
- **Digital root**: repeated reduction to one digit (e.g., $137 \to 11 \to 2$).

3.2 Indexed candidate retrieval

To avoid linear scans over large databases, the reference set is indexed by:

- gematria,
- reduction,
- digital root,
- and POS-crossed versions of those indices when POS matching is enabled.

3.3 Strict vs fluid gematria behavior

The current logic supports two modes:

- **Fluid gematria OFF**: strict word-level gematria is enforced; candidates are filtered to exact gematria matches only.
- **Fluid gematria ON**: reduction/root expansions are allowed, and jump exploration may broaden the search radius.

3.4 POS matching and fluid POS

When POS matching is enabled, retrieval is POS-conditioned.

- **Fluid POS OFF**: seed POS tags are frozen from the original sentence word slots.
- **Fluid POS ON**: POS is recomputed dynamically from current word forms.

To avoid stagnation in narrow POS buckets, logic includes fallback broadening if no non-self alternatives exist.

3.5 Candidate scoring and selection

For a source word s and candidate c, the base structural score is composed from gematria and numerological distances:

$$\Delta_G = \frac{|G(c) - G(s)|}{1000}, \quad \Delta_R = \frac{|R(c) - R(s)|}{9}, \quad \Delta_D = \frac{|D(c) - D(s)|}{9},$$

$$\text{base}(s, c) = \Delta_G + 0.35\Delta_R + 0.35\Delta_D.$$

If LTM probabilities are available, a blended score is used:

$$\text{blend}(s, c) = (1 - \lambda)\,\text{base}(s, c) + \lambda\,(1 - p_{\text{ltm}}(c|\text{context})),$$

where $\lambda \in [0, 1]$ is `ltm_weight` and p_{ltm} can be reduced by a common-word penalty.

Selection combines ranked deterministic choice with controlled stochasticity (top-k exploration and optional jump mode).

3.6 Sentence iteration and convergence guard

During each logic phase, words with activation magnitude above threshold are eligible for replacement. Iteration is bounded by:

- maximum iteration count,
- and cycle detection via previously seen sentence states.

This prevents infinite loops and gives practical convergence behavior.

3.7 Audio synthesis algorithm

The sentence is converted to note sequences and rendered as looped audio:

- waveform choice (sine, triangle, square, sawtooth, noise-driven modes),
- frequency mapping mode (original, Pythagorean, equal-tempered modal variants),
- polyphonic voice count/spread,
- ADSR envelope,
- minimum note duration and duration scaling.

Chapter 4

Gematria and Numerology in Historical Perspective

4.1 From ancient number-symbolism to formal systems

Number symbolism appears across many ancient cultures, including Egypt and Mesopotamia, and later in Greek philosophical traditions where number harmony was treated as a structural principle of cosmos and proportion. In late antiquity and medieval religious-intellectual contexts, alphabetic-number correspondences became increasingly formalized in scriptural interpretation traditions.

4.2 Hebrew Kabbalah and numerological interpretation

In Hebrew kabbalistic practice, letter-number correspondences support interpretive techniques such as gematria, where numerical values of words are compared to identify conceptual parallels. Within this worldview, language is not only semantic but also structural-symbolic.

4.3 Hermetic Qabalah (based on `chicken_qabalah.txt`)

The source text `chicken_qabalah.txt` frames "Chicken Qabalah" as a practical adaptation of Hebrew Qabalah for Western Hermetic practitioners. Several themes are emphasized:

- The distinction *Kabbalah/Cabala/Qabalah* is presented as partly traditional/orthodox vs practical/esoteric usage.
- The intended audience includes Western Hermetic domains (Tarot, ceremonial magick, Rosicrucianism, alchemy, esoteric Freemasonry, etc.).
- Practice is centered on letter meanings and numbers, the Tree of Life, and related meditative/symbolic exercises.
- The *Sepher Yetzirah* is highlighted as foundational, particularly for the concepts of the ten *Sephiroth* and the symbolic role of alphabetic creation.

This Hermetic framing is directly relevant to SanaVerkko because the software operationalizes a pragmatic, transformation-oriented form of gematria rather than a historical-philological reconstruction.

Chapter 5

Part-of-Speech (POS) Tagging

5.1 POS in NLP

POS tagging assigns grammatical categories (noun, verb, adjective, adverb, etc.) to word tokens. In modern NLP, POS can be estimated by rule-based systems, statistical sequence models, or neural contextual models. POS constraints are useful when one wants substitutions that preserve grammatical role.

5.2 POS implementation in SanaVerkko

SanaVerkko uses a hybrid strategy:

- If NLTK tagger resources are available, it uses NLTK POS tagging.
- Otherwise, it falls back to a suffix-based heuristic:
  - `-ly` → adverb,
  - `-ing`/`-ed` → verb,
  - adjective-like suffixes (`-ous`, `-ful`, `-ive`, ...),
  - noun-like suffixes (`-ness`, `-tion`, `-ment`, ...),
  - default noun.

The system caches POS tags and supports both frozen-seed POS and fluid POS mutation modes.

Chapter 6

Neural Language Models: General and SanaVerkko-Specific

6.1 General overview

A neural language model estimates token probabilities conditioned on context:

$$P(w_t \mid w_{t-k}, \ldots, w_{t-1}).$$

Classic families include feed-forward n-gram-style MLPs, RNN/LSTM models, and Transformer-based architectures. Key training objective is usually cross-entropy minimization over next-token prediction.

6.2 SanaVerkko LTM model architecture

The model in `sv_ltm.py` is a compact next-word predictor:

1. Character-level encoder (Char-CNN) builds a fixed feature vector for each word.
2. Context window (default size 3) concatenates recent word feature vectors.
3. MLP head with two hidden layers projects to vocabulary logits.
4. Softmax yields next-word probabilities.

6.3 Training objective

The training objective is cross-entropy with L2 regularization:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log p(y_i \mid x_i) + \frac{\lambda}{2}\left(\|W_1\|_2^2 + \|W_2\|_2^2 + \|W_3\|_2^2\right).$$

6.4 Inference-time integration in SanaVerkko

The core app queries LTM probabilities for candidate words and blends them into ranking with a tunable weight. This means LTM does not replace gematria logic; it re-weights candidate preference.

6.5 GPU acceleration path

Phase-1 acceleration is implemented with optional PyTorch backend:

- `device=auto|cpu|mps|cuda` for training,
- Metal (MPS) support on Apple Silicon when available,
- CPU fallback when PyTorch or requested backend is unavailable.

At runtime, backend status can be displayed in the app LTM status label.

Chapter 7

Discussion and Future Directions

7.1 Strengths

- Clear symbolic core (gematria + numerological transforms).
- Practical controllability via strict/fluid toggles and exploration parameters.
- Efficient indexed retrieval for large reference databases.
- Optional neural prior (LTM) without losing interpretable symbolic behavior.

7.2 Limitations

- Linguistic quality depends on reference corpus and POS coverage.
- Heuristic POS fallback is intentionally simple.
- LTM is a compact model and not as expressive as large transformer LMs.
- Numerological mappings are symbolic/interpretive, not scientific semantics.

7.3 Potential extensions

- Richer POS/morphology constraints.
- Subword or transformer-based optional LTM.
- Better observability dashboards for candidate pools and fallback reasons.
- Preset metadata and experiment reproducibility tooling.

Source Notes

- Implementation basis: `sanaVerkkoCore.py`, `sv_ltm.py`, `sanasyna.py` in this repository.
- Hermetic Qabalah framing basis: `chicken_qabalah.txt` sections discussing practical Western Hermetic usage, Tree of Life focus, and *Sepher Yetzirah* context.